

Vision Inspector

A transparent computer-vision image inspection dashboard

Course	Computer Vision
Submission	VITyarthi Build Your Own Project
Version	1.0.0

1. Introduction

Vision Inspector is a local-first tool that turns an image into a compact, interpretable inspection report. It applies classical computer-vision operations so students and dataset curators can understand why an image may need review without relying on an opaque model or external API.

2. Problem Statement

Image collections commonly contain exposure problems, blur, weak contrast, repetitive colors, or insufficient structure. Manual review is slow and subjective. The proposed system provides a repeatable first-pass audit while leaving the final retain/correct/review decision to a human.

Objectives

- Apply image statistics, color-space conversion, clustering, edge detection, contour extraction, and connected components.
- Deliver three major functional modules with clear input/output contracts.
- Provide both an interactive dashboard and a JSON-capable CLI.
- Document the design and validate it with automated tests.

3. Functional Requirements

FR-1 Quality	Calculate brightness, contrast, Laplacian blur score, a 0–100 heuristic quality score, and warnings.
FR-2 Color	Estimate colorfulness, mean RGB, and the dominant RGB palette using deterministic k-means.
FR-3 Structure	Calculate Canny edge density, external contour count, largest contour ratio, and connected components.
FR-4 Output	Show human-readable findings in Streamlit and serialize the complete report as JSON from the CLI.

4. Non-functional Requirements

- Performance: downsample color clustering to 80×80; avoid unnecessary full-resolution operations.
- Security: process uploaded images locally; never transmit them to a third party.
- Usability: provide plain-language warnings and a one-command CLI path.
- Reliability: reject unreadable files with an actionable ValueError.
- Maintainability: keep modules single-purpose and expose typed dataclass results.
- Resource efficiency: use bounded palette samples and no persistent database.

5. System Architecture

The architecture separates presentation, orchestration, domain algorithms, and result contracts.

User	Uploads a PNG/JPEG in the dashboard or supplies a local path to the CLI.
App / CLI	Accepts input and requests one analysis through the pipeline.
Pipeline	Loads and validates the image, then invokes quality, color, and structure modules.
Analysis modules	OpenCV and NumPy algorithms return typed, serializable result objects.
Report	Dashboard cards or JSON output provide findings and measurements.

6. Design Diagrams

Workflow: Input image → Load/validate → Quality + Color + Structure (parallel conceptual modules) → Combine report → Display/export.

Use cases

- Dataset curator: upload image, inspect warnings, decide whether to review.
- Developer: run CLI with --json, pipe metrics into another tool.
- Student: read each metric and connect it to a classical CV operation.

Component design

The pipeline depends on three pure analysis modules. Each returns a dataclass defined in models.py. The UI and CLI depend on the pipeline, not on OpenCV details. This dependency direction makes algorithms testable without the interface.

Storage design

No persistent storage is used. The source image remains at its original location; the report exists in memory and can be serialized to stdout. This reduces privacy risk and keeps the project portable.

7. Implementation Details

Quality uses grayscale mean/std statistics and Laplacian variance. Color analysis converts BGR to HSV for saturation-based colorfulness and uses deterministic OpenCV k-means on a bounded thumbnail. Structure analysis uses Canny edges, external contours, Otsu thresholding, and connected components. The Streamlit app creates a temporary local copy only for the duration of an inspection.

8. Testing Approach

The test suite writes a synthetic two-color fixture, checks dimensions and bounded metric ranges, and verifies an invalid file produces an actionable error. The design supports further fixture-based tests for dark, bright, and blurry images.

Test	Expected result
------	-----------------

Synthetic valid PNG	Report dimensions and metrics are valid
Unreadable text file	ValueError includes supported image
CLI --json	Valid JSON report on stdout

9. Challenges Faced

The main design challenge was balancing useful signals with explainability. A single score can hide the cause of a problem, so the project reports both the score and its component metrics. Another challenge was keeping execution deterministic and lightweight; bounded sampling and fixed clustering criteria address this.

10. Learnings and Key Takeaways

- Classical CV primitives can produce useful dataset triage signals without model training.
- Typed boundaries make it easier to reuse the same analysis from a UI and a CLI.
- Human-readable warnings are most useful when paired with the underlying measurement.

11. Future Enhancements

- Batch directory analysis with CSV/JSONL export.
- Optional perceptual-hash duplicate detection.
- Configurable thresholds and saved inspection profiles.
- Interactive plots and side-by-side before/after enhancement previews.

12. References

OpenCV documentation: image processing, color conversion, k-means, Canny, contours, and connected components. NumPy documentation: array statistics and reshaping. Streamlit documentation: file upload and metric components.

13. Screenshots / Results

Runtime verification was performed against the Streamlit dashboard using the generated sample_scene.png. The working dashboard displayed the uploaded image, quality score 73.83/100, brightness 158.08, edge density 1.36%, colorfulness 48.95%, and a no-warning success state. The three module tabs rendered quality, color palette, and structure outputs. Runtime captures are documented in the submission notes and should be saved into docs/screenshots before portal upload if local screenshot export is required.

14. Manual submission fields

Student name, roll number, faculty, semester, and academic year were not supplied and remain intentionally absent from this generic Computer Vision report cover page.